

Aula 10

DataPrev (Perfil 3: Desenvolvimento de Software) Desenvolvimento de Software - 2026 (Pós-Edital)

Autor:
Vinicius Borges

09 de Julho de 2026

Índice

1) Automação de Testes - JUnit - Teoria	3
2) Automação de Testes - JUnit - Questões Comentadas	14
3) Automação de Testes - JUnit - Lista de Questões	20

JUNIT

O **JUnit** é um framework open-source, de teste de unidade para a linguagem de programação Java, criado por Erich Gamma e Kent Beck, com suporte à criação de testes automatizados na linguagem de programação Java.

De acordo com **Martin Fowler**, um dos primeiros **adeptos do framework**, o JUnit nasceu em um voo de Zurique para o OOPSLA de 1997 em Atlanta. Kent estava voando com Erich Gamma, e o que mais dois geeks fariam em um longo voo além de programar? A primeira versão do JUnit foi construída lá, programada em pares e feita o teste primeiro (uma forma agradável de geekery meta-circular).

JUNIT É UM FRAMEWORK DE TESTE DE UNIDADE PARA ESCREVER E EXECUTAR TESTES AUTOMATIZADOS REPETÍVEIS EM JAVA

Esse framework **facilita a criação e manutenção do código para a automação de testes com apresentação dos resultados**. Com ele, pode ser verificado se **cada método de uma classe funciona da forma esperada**, exibindo possíveis erros ou falhas, podendo ser utilizado tanto para a execução de baterias de testes como para extensão.

O JUnit automatiza o teste de unidade e reduz o esforço necessário para testar o código com frequência durante o desenvolvimento.

(CESPE – AJ STJ– 2015) JUnit é um framework utilizado para facilitar a geração de testes a fim de se verificar se os resultados gerados pelos métodos escritos em Java são os esperados.

Comentários: O JUnit facilita a criação e manutenção do código para a automação de testes com apresentação dos resultados. Com ele, pode ser verificado se cada método de uma classe funciona da forma esperada, exibindo possíveis erros ou falhas podendo ser utilizado tanto para a execução de baterias de testes como para extensão. (Gabarito: Correto).

Com JUnit, o programador tem a possibilidade de usar esta ferramenta para criar um modelo padrão de testes, muitas vezes de forma automatizada.

O **teste de unidade** testa o **menor dos componentes** de um sistema de maneira isolada. Cada uma dessas unidades define um conjunto de estímulos (chamada de métodos), e de dados de entrada e saída associados a cada estímulo. As entradas são parâmetros e as saídas são o valor de retorno, exceções ou o estado do objeto. Tipicamente um teste unitário executa um método individualmente e compara uma saída conhecida após o processamento dela.

(Quadrix – 2018) O JUnit é um método de testes de software do padrão xUnit voltado para sistemas desenvolvidos em linguagem Java.

Comentários: O JUnit é um framework de código aberto para automação de testes unitários desenvolvido para a linguagem Java. Este framework tem por base o xUnit, que é o nome dado a uma família de frameworks de testes para diversas plataformas. (Gabarito: Correto).

JUnit tem sido importante no desenvolvimento de **desenvolvimento orientado a testes** e é uma família de estruturas de teste de unidade que é coletivamente conhecida como **xUnit** que se originou com SUnit. Algumas vantagens de se utilizar JUnit:

- Permite a criação rápida de código de teste enquanto possibilita um aumento na qualidade do sistema sendo desenvolvido e testado;
- Não é necessário escrever o próprio framework;
- Amplamente utilizado pelos desenvolvedores da comunidade código-aberto, possuindo muitos exemplos;
- Uma vez escritos, os testes são executados rapidamente sem que, para isso, seja interrompido o processo de desenvolvimento;
- JUnit checa os resultados dos testes e fornece uma resposta imediata;
- Pode-se criar uma hierarquia de testes que permitirá testar apenas uma parte do sistema ou todo ele;
- Escrever testes com JUnit permite que o programador perca menos tempo depurando seu código;
- JUnit é LIVRE.

A experiência adquirida com o JUnit tem sido importante na consolidação do **Test Driven Development – TDD** (Desenvolvimento Dirigido a Testes). Além disso, ele foi adaptado a outras linguagens, tais como C# (NUnit), Python, Fortran, e C++.

Uma pesquisa realizada em 2013 em 10.000 projetos Java hospedados no GitHub descobriu que **JUnit** (empacada com slf4j-api) era a **biblioteca externa mais comumente incluída**. Cada biblioteca foi utilizada por 30,7% dos projetos.

Como efeito colateral de seu amplo uso, as versões anteriores do JUnit continuam populares, com o JUnit 4 tendo mais de **100.000 usos por outros componentes de software** no repositório central do Maven.

Um dispositivo de teste **JUnit** é um **objeto Java**. Os métodos de teste devem ser anotados pela anotação **@Test**. Se a situação exigir, também é possível **definir um método para executar antes (ou depois) de cada (ou todos) os métodos de teste** com anotações **@BeforeEach (ou @AfterEach)** e **@BeforeAll (ou @AfterAll)**.

@BeforeEach é executado antes de cada teste, nos permite inicializar objetos ou realizar qualquer tarefa relacionada a preparação de um ambiente adequado para que os objetos reunidos em um contexto executem as suas ações de forma consistente e isolada.

@AfterEach desfaz esse ambiente, removendo da memória caches, fechando streams ou quaisquer outros dados que poderiam comprometer a confiabilidade do próximo teste.

Na JUnit 4, as anotações para retornos de chamada de execução de teste eram **@BeforeClass**, **@Before**, **@After** e **@AfterClass**, em oposição a **@BeforeAll**, **@BeforeEach**, **@AfterEach** e **@AfterAll** da JUnit 5.

Pessoal, vejamos as atualizações e os comandos obsoletos do JUnit 4. Abaixo, na tabela, vamos ver a esquerda os comandos obsoletos, ou seja, comandos que já não existem. Já a direita os comandos que devem ser usados em substituição aos comandos obsoletos.

JUnit 4	JUnit Jupiter
@Before	@BeforeEach
@After	@AfterEach
@BeforeClass	@BeforeAll
@AfterClass	@AfterAll
@Ignore	@Disabled
@Category	@Tag
@RunWith	@ExtendWith
@Rule	@ExtendWith
@ClassRule	@RegisterExtension

Vejamos a descrição das anotações do **JUnit 5**.

Anotação	Descrição
@Test	Indica que um método é um método de teste.
@ParameterizedTest	Indica que um método é um teste parametrizado. Esses métodos são herdados, a menos que sejam substituídos.
@RepeatedTest	Indica que um método é um modelo de teste para um teste repetido. Esses métodos são herdados, a menos que sejam substituídos.
@TestFactory	Indica que um método é uma fábrica de testes para testes dinâmicos. Esses métodos são herdados, a menos que sejam substituídos.

@TestTemplate	Indica que um método é um modelo para casos de teste projetados para serem invocados várias vezes, dependendo do número de contextos de invocação retornados pelos provedores registrados. Esses métodos são herdados, a menos que sejam substituídos.
@TestClassOrder	Usado para configurar a ordem de execução da classe de @Nestedteste para classes de teste na classe de teste anotada. Essas anotações são herdadas.
@TestMethodOrder	Usado para configurar a ordem de execução do método de teste para a classe de teste anotada; semelhante ao JUnit 4 @FixMethodOrder. Essas anotações [] herdadas.
@TestInstance	Usado para configurar o ciclo de vida da instância de teste para a classe de teste anotada. Essas anotações {} são herdadas.
@DisplayName	Declara um nome de exibição personalizado para a classe de teste ou método de teste. Essas anotações não são herdadas.
@DisplayNameGeneration	Declara um gerador de nome de exibição personalizado para a classe de teste. Essas anotações são herdadas.
@BeforeEach	Indica que o método anotado deve ser executado antes de cada método @Test, @RepeatedTest, @ParameterizedTest ou @TestFactory na classe atual; análogo ao @After do JUnit 4. Esses métodos são herdados – a menos que sejam sobrepostos ou substituídos (ou seja, substituídos com base apenas na assinatura, independentemente das regras de visibilidade do Java).
@AfterEach	Indica que o método anotado deve ser executado após cada método @Test, @RepeatedTest, @ParameterizedTest ou @TestFactory na classe atual; análogo ao @After do JUnit 4. Esses métodos são herdados – a menos que sejam sobrepostos ou substituídos (ou seja, substituídos com base apenas na assinatura, independentemente das regras de visibilidade do Java).

@BeforeAll	Indica que o método anotado deve ser executado antes de todos os métodos @Test, @RepeatedTest, @ParameterizedTeste @TestFactory na classe atual; análogo ao @BeforeClass JUnit 4. Esses métodos são herdados – a menos que sejam ocultos, sobrepostos ou substituídos (ou seja, substituídos com base apenas na assinatura, independentemente das regras de visibilidade do Java) – e devem ser, a menos que o ciclo de vida da instância de testestatic "por classe" seja usado.
@AfterAll	Indica que o método anotado deve ser executado após todos os @Test métodos, @RepeatedTest, @ParameterizedTeste @TestFactoryna classe atual; análogo ao JUnit 4 @AfterClass. Esses métodos são herdados – a menos que sejam ocultos, sobrepostos ou substituídos (ou seja, substituídos com base apenas na assinatura, independentemente das regras de visibilidade do Java) – e devem ser, a menos que o ciclo de vida da instância de testestatic "por classe" seja usado.
@Nested	Indica que a classe anotada é uma classe de teste aninhada não estática. No Java 8 até o Java 15, os métodos @BeforeAll e @AfterAll não podem ser usados diretamente em uma classe de teste, a menos que o ciclo de vida da instância de teste @Nested "por classe" seja usado. A partir do Java 16, e os métodos podem ser declarados como em uma classe de teste com qualquer modo de ciclo de vida da instância de teste. Essas anotações não são <i>herdadas</i> .@BeforeAll @AfterAllstatic @Nested
@Tag	Usado para declarar tags para testes de filtragem, seja no nível de classe ou método; análogo aos grupos de teste no TestNG ou Categorias no JUnit 4. Essas anotações são herdadas no nível da classe, mas não no nível do método.
@Disabled	Usado para desabilitar uma classe de teste ou método de teste; análogo ao JUnit 4 @Ignore. Essas anotações não são herdadas.
@Timeout	Usado para falhar em um teste, fábrica de teste, modelo de teste ou método de ciclo de vida, se sua execução exceder uma determinada duração. Essas anotações são herdadas.

@ExtendWith	Usado para registrar extensões declarativamente. Essas anotações são herdadas.
@RegisterExtension	Usado para registrar ramais programaticamente por meio de campos. Esses campos são herdados, a menos que sejam sombreados.
@TempDir	Usado para fornecer um diretório temporário via injeção de campo ou injeção de parâmetro em um método de ciclo de vida ou método de teste; localizado no <code>org.junit.jupiter.api.io.pacote</code> .
@EnabledIf/@DisabledIf	Usado para sinalizar que a classe de teste ou método de teste anotado está habilitado somente se a condição fornecida for avaliada como <code>true</code> . Quando aplicado no nível de classe, todos os métodos de teste dessa classe serão desabilitados na mesma condição.
@EnabledOnOs/@DisabledOnOs	Usado para sinalizar que a classe de teste ou método de teste anotado está habilitado/desabilitado apenas em um ou mais sistemas operacionais especificados. Quando aplicado no nível de classe, todos os métodos de teste dessa classe serão habilitados nos mesmos sistemas operacionais especificados.
@EnabledIfSystemProperty/ DisabledIfSystemProperty	Usado para sinalizar que a classe de teste ou método de teste anotado só está ativado/desativado se o valor da propriedade de sistema especificada corresponder à expressão regular especificada. Quando declarado no nível da classe, o resultado também será aplicado a todos os métodos de teste dessa classe.
@DisabledIfEnvironmentVariable/ @EnabledIfEnvironmentVariable	Usado para sinalizar que a classe de teste ou método de teste anotado está habilitado/desabilitado se o valor da variável de ambiente especificada corresponder à expressão regular especificada.

(CESPE – DP DF– 2022) Na definição de métodos de teste em JUnit, a anotação `@BeforeClass` pode ser usada em métodos que implementem atividades que consomem muito tempo.

Comentários: Pessoal, `@BeforeClass` foi substituída por `@BeforeAll` e trata-se de uma anotação que indica que o método anotado deve ser executado antes de todos os métodos. Como `@BeforeAll` pode ser inserida em qualquer método, é possível aplicar `@BeforeAll` a métodos que implementam atividades que consomem muito tempo. (Gabarito: Correto).

Na JUnit 3, os acessórios de teste tinham que herdar de `junit.framework.TestCase`. Além disso, os métodos de teste tiveram que ser prefixados com 'teste'

JUnit 5 é a nova geração do **JUnit**. O objetivo é **criar uma base atualizada para testes** do lado do desenvolvedor na JVM. Isso inclui focar no Java 8 e superior, bem como habilitar muitos estilos diferentes de teste. JUnit 5 é o resultado do **JUnit Lambda** e sua campanha de crowdfunding no Indiegogo.

Ao contrário das versões anteriores do JUnit, o **JUnit 5** é composto por **vários módulos** diferentes de três subprojetos diferentes. **JUnit 5 = Plataforma JUnit + JUnit Jupiter + JUnit Vintage**

JUNIT 5 = PLATAFORMA JUNIT + JUNIT JUPITER + JUNIT VINTAGE

A Plataforma JUnit serve como base para lançar estruturas de teste na JVM. Ele também define a `TestEngineAPI` para desenvolver uma estrutura de teste que é executada na plataforma. Além disso, a plataforma fornece um Console Launcher para iniciar a plataforma a partir da linha de comando e o JUnit Platform Suite Engine para executar um conjunto de testes personalizado usando um ou mais mecanismos de teste na plataforma.

O suporte de primeira classe para a plataforma JUnit também existe em IDEs populares (IntelliJ IDEA, Eclipse, NetBeans e Visual Studio Code) e ferramentas de compilação (Gradle, Maven...).

JUnit Jupiter é a combinação do **modelo de programação e do modelo de extensão** para escrever testes e extensões no **JUnit 5**. O subprojeto Jupiter fornece um `TestEngine` para executar testes baseados em Jupiter na plataforma.

JUnit Vintage fornece um **TestEngine** para executar testes baseados em JUnit 3 e JUnit 4 na plataforma. Requer que o JUnit 4.12 ou posterior esteja presente no caminho da classe ou no caminho do módulo.

JUnit 5 requer Java 8 (ou superior) em tempo de execução. No entanto, você ainda pode testar o código que foi compilado com versões anteriores do JDK.

(CESPE – PGDF– 2021) O JUnit 5 é formado por JUnit Platform, JUnit Jupiter e JUnit Vintage; o JUnit Jupiter pode ser utilizado em programas escritos em Kotlin.

Comentários: Pessoal, ao contrário das versões anteriores do JUnit, o JUnit 5 é composto por vários módulos diferentes de três subprojetos diferentes. JUnit 5 = Plataforma JUnit + JUnit Jupiter + JUnit Vintage. (Gabarito: Correto)

Falando de forma simples e direta, um teste unitário é um teste realizado para **verificar a funcionalidade de um determinado trecho de código**, verificar se ele realmente faz o que se propõe a fazer. O objetivo de testes unitários **não é testar toda a funcionalidade do sistema**, ou a integração de várias partes do sistema de uma única vez, mas **realizar testes isolados, testando blocos específicos** do sistema, mais comumente os métodos das classes.

Por padrão, as **classes e métodos de teste serão ordenados usando um algoritmo determinístico**, mas intencionalmente não óbvio. Isso garante que **execuções subsequentes de um conjunto de testes executem classes de teste e métodos de teste na mesma ordem**, permitindo **compilações repetíveis**.

Embora os verdadeiros testes de unidade normalmente **não devam depender da ordem em que são executados**, há momentos em que é necessário impor uma ordem específica de execução do método de teste - por exemplo, ao escrever testes de integração ou testes funcionais em que a sequência dos testes é importante, especialmente em conjunto com `@TestInstance(Lifecycle.PER_CLASS)`.

Para **controlar a ordem** na qual os métodos de teste são executados, anote sua classe de teste ou interface de teste **@TestMethodOrder** e especifique a `MethodOrderer` implementação desejada. Você pode implementar seu próprio `MethodOrderer` personalizado ou usar uma das seguintes implementações internas `MethodOrderer`.

- **MethodOrderer.DisplayName**: classifica os métodos de teste alfanumericamente com base em seus nomes de exibição
- **MethodOrderer.MethodName**: classifica os métodos de teste alfanumericamente com base em seus nomes e listas de parâmetros formais
- **MethodOrderer.OrderAnnotation**: classifica os métodos de teste numericamente com base nos valores especificados por meio da `@Order` anotação
- **MethodOrderer.Random**: ordena os métodos de teste de forma pseudo-aleatória e suporta a configuração de uma semente personalizada
- **MethodOrderer.Alphanumeric**: classifica os métodos de teste alfanumericamente com base em seus nomes e listas de parâmetros formais; preterido em favor de `MethodOrderer.MethodName`, que será removido no JUnit 6.0

(CESPE – BACEN – 2013) O JUnit é um framework que permite realizar testes unitários de códigos Java, herdando algumas características da própria linguagem. Na versão 4.11, ele permite, entre outros recursos, mudar a ordem de testes, criar testes parametrizados, executar testes de exceção e testes temporizados por meio da palavra-chave timeout.

Comentários: Questão mais elaborada, cobra a especificidade do JUnit 4.11. de fato, a partir da versão 4.11 do JUnit, nós podemos especificar um padrão de ordenação a ser seguido na execução dos testes. (Gabarito: Letra A).

Toda classe de teste é uma subclasse da classe **TestCase** que é declarada no pacote do JUnit.framework localizado no arquivo JUnit.jar. TestCase é a classe que chama os métodos das classes para realizar os testes. Podendo ser realizado um único teste ou vários testes ao mesmo tempo, através do objeto composto TestSuite.

TODA CLASSE DE TESTE É UMA SUBCLASSE DA CLASSE TESTCASE

Há as asserções, que tem o objetivo de verificar valores de saída de um processamento, facilitando assim a análise dos resultados obtidos em testes de unidade. Ao executar o JUnit as saídas esperadas são validadas através dos métodos chamados assertXxx, onde Xxx pode ser True, False Equals, ou outra condição. Além disso, pode ser passada para esses métodos alguma mensagem específica caso o teste falhe.

Convenções

São consideradas as seguintes convenções do JUnit:

- Test Case Class: [NomeClasse]Test.java, onde NomeClasse é o nome da classe a ser testada.
- Test Case Method: [NomeMetodo], onde NomeMetodo é o nome do método a ser testado.

Estas convenções são consideradas **boas práticas em testes**, assim como separar o código de teste do código da aplicação em si. Isto pode ser feito criando pacotes com nomes referenciando testes, onde deverão estar somente as classes de teste.

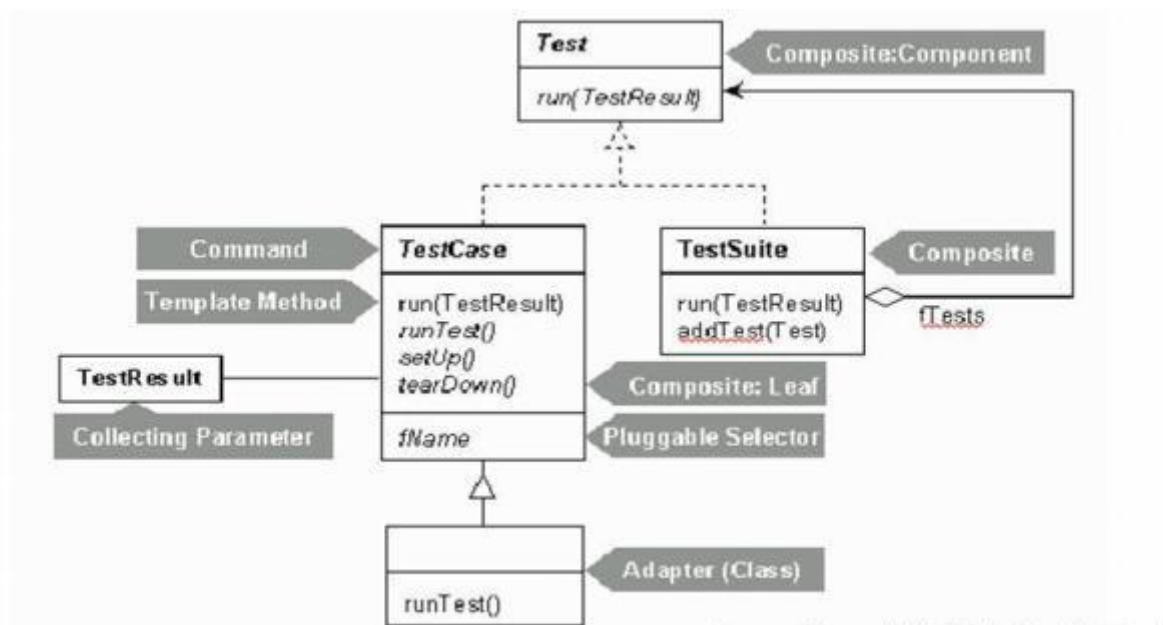
Com o JUnit, pode ser verificado se cada método de uma classe funciona da forma esperada, exibindo possíveis erros ou falhas, podendo ser utilizado tanto para a execução de baterias de testes como para extensão.

Uma vez que todas as condições testadas obtiveram o resultado esperado após a execução do teste, considera-se que a classe ou método está de acordo com a especificação, incrementando a

qualidade daquela unidade. Com JUnit, o programador tem uma ferramenta muito poderosa que o ajudará a eliminar todos (ou quase todos) os bugs de seu código de maneira mais atraente. Ou seja, os programadores gostam de programar, então se criou uma forma interessante de realizar testes onde é possível a criação de programas que realizem os testes pelo programador.

É utilizando esse conceito que JUnit permite deixar a fase de teste de unidade bem mais agradável ao programador. Para utilizar o JUnit, é necessário criar uma classe que estenda `junit.framework.TestCase`. A partir daí, para cada método a ser testado é necessário definir métodos, os quais devem ser públicos e sem retorno de argumentos.

Métodos `testeXxx()` na classe de teste devem ser `public void` e não podem receber nenhum parâmetro. Eles criam um objeto definindo o ambiente de teste, executam esses testes utilizando o ambiente que foi criado e verificam os resultados que podem terminar, falhar ou provocar uma exceção.



Vantagens de se utilizar JUnit

- Permite a criação rápida de código de teste enquanto possibilita um aumento na qualidade do sistema sendo desenvolvido e testado;
- Não é necessário escrever o próprio framework;
- Amplamente utilizado pelos desenvolvedores da comunidade código-aberto, possuindo muitos exemplos;
- JUnit é elegante e simples. Quando testar um programa se torna algo complexo e demorado, então não existe motivação para o programador fazê-lo;
- Uma vez escritos, os testes são executados rapidamente sem que, para isso, seja interrompido o processo de desenvolvimento;
- JUnit verifica os resultados dos testes e fornece uma resposta imediata;

- Pode-se criar uma hierarquia de testes que permitirá testar apenas uma parte do sistema ou todo ele;
- Escrever testes com JUnit permite que o programador perca menos tempo depurando seu código;
- Todos os testes criados utilizando o JUnit são escritos em Java;
- JUnit é LIVRE.

O JUnit possui uma grande integração com outras ferramentas de desenvolvimento, como Jbuilder, Kawa, Jdeveloper, entre outros. Além disto, foram projetadas as extensões do JUnit voltados para diversos segmentos como banco de dados, XML, J2EE e WEB.

Como os testes são realizados?

A primeira anotação a ser utilizada para criar testes unitários é **@Test**, que informa ao JUnit quais são os métodos de teste de uma classe. Para o JUnit o nome do método não importa, pois desde que ele seja anotado dessa maneira ele será identificado como um método de teste.

Uma vez que cada teste deve ser executado de forma isolada e independente, a fim de garantir que eles não sofram com a ausência de dependências ou a sobra de instâncias de objetos usados em testes anteriores, é importante que saibamos utilizar os mecanismos disponíveis para a criação e destruição de ambientes isolados de teste.

QUESTÕES COMENTADAS - JUNIT

1. (CESPE – DP DF– 2022) Julgue o item a seguir, acerca de CSS3, JMS, JSON e JUnit.

Na definição de métodos de teste em JUnit, a anotação `@BeforeClass` pode ser usada em métodos que implementem atividades que consomem muito tempo.

Comentários:

Pessoal, `@BeforeClass` foi substituída por `@BeforeAll` e trata-se de uma anotação que indica que o método anotado deve ser executado antes de todos os métodos. Como `@BeforeAll` pode ser inserida em qualquer método, é possível aplicar `@BeforeAll` a métodos que implementam atividades que consomem muito tempo.

Gabarito: Correto

2. (CESPE – PGDF– 2021) Julgue o item seguinte, a respeito de JMS (Java Message Service), JUnit e Spring Framework.

O JUnit 5 é formado por JUnit Platform, JUnit Jupiter e JUnit Vintage; o JUnit Jupiter pode ser utilizado em programas escritos em Kotlin.

Comentários:

Ao contrário das versões anteriores do JUnit, o JUnit 5 é composto por vários módulos diferentes de três subprojetos diferentes. JUnit 5 = Plataforma JUnit + JUnit Jupiter + JUnit Vintage

Gabarito: Correto

3. (CESPE – TJ PA– 2020) Assinale a opção que apresenta a anotação que pode ser utilizada no JUnit para condicionar a execução de um teste a partir da avaliação de um script escrito em JavaScript.

- a) `@EnabledOnOs`.
- b) `@EnabledOnJre`
- c) `@DisabledIfSystemProperty`.
- d) `@DisabledIfEnvironmentVariable`
- e) `@EnabledIf`

Comentários:

Vejam as definições das alternativas: `@EnabledIf/@DisabledIf`: Usado para sinalizar que a classe de teste ou método de teste anotado está habilitado somente se a condição fornecida for avaliada como true. Quando aplicado no nível de classe, todos os métodos de teste dessa classe serão desabilitados na mesma condição. `@EnabledOnOs/@DisabledOnOs`: Usado para sinalizar que a

classe de teste ou método de teste anotado está habilitado/desabilitado apenas em um ou mais sistemas operacionais especificados. Quando aplicado no nível de classe, todos os métodos de teste dessa classe serão habilitados nos mesmos sistemas operacionais especificados. `@EnabledIfSystemProperty/ DisabledIfSystemProperty`: Usado para sinalizar que a classe de teste ou método de teste anotado só está ativado/desativado se o valor da propriedade de sistema especificada corresponder à expressão regular especificada. Quando declarado no nível da classe, o resultado também será aplicado a todos os métodos de teste dessa classe. `@DisabledIfEnvironmentVariable/@EnabledIfEnvironmentVariable`: Usado para sinalizar que a classe de teste ou método de teste anotado está habilitado/desabilitado se o valor da variável de ambiente especificada corresponder à expressão regular especificada. Pessoal, nosso gabarito é `@EnabledIf/@DisabledIf`: Usado para sinalizar que a classe de teste ou método de teste anotado está habilitado somente se a condição fornecida (script) for avaliada como true, ou seja, ele condiciona a execução de um teste a partir da avaliação de um script.

Gabarito: Letra E

4. (CESPE – COGE CE – 2019) Nos testes unitários, é possível testar uma classe ou até mesmo um objeto Java. Nesse contexto, uma classe, para ser testada pelo Junit, deve se estender a partir de
- a) Collection.
 - b) TestExecution.
 - c) TestSetUp.
 - d) TestCase
 - e) TestReporter.

Comentários: Toda classe de teste é uma subclasse da classe TestCase.

Gabarito: Letra D

5. (CESPE – SLU-DF – 2019) Na prática de integração contínua para desenvolvimento de software, vários colaboradores criam e mantêm o código de forma organizada e controlada, utilizando ferramentas como Git (controle de versão), Junit (testes), Hudson e Jenkins (deploys em ambientes de desenvolvimento e produção), o que reduz a geração de erros de integração.

Comentários: Na prática de desenvolvimento de software denominada integração contínua, os membros de um time integram seu trabalho frequentemente — geralmente cada pessoa integra pelo menos diariamente —, podendo haver múltiplas integrações por dia. Cada integração é verificada por um build automatizado (incluindo testes) para detectar erros de integração o mais rápido possível. Muitos times acham que essa abordagem leva a uma significativa redução nos problemas de integração e permite que um time desenvolva um software coeso mais rapidamente. Para isso, são utilizadas ferramentas como Git (controle de versão), JUnit (testes), Hudson e Jenkins (deploys em ambientes de desenvolvimento e produção).

Gabarito: Correto

6. (CESPE – TJ STJ– 2018) Uma característica e limitação do JUnit é a impossibilidade de definição de parâmetros para construtores e métodos.

Comentários: Pessoal, na verdade a banca colocou uma “impossibilidade” no meio da questão para torná-la incorreta. É possível sim, a definição de parâmetros para construtores e métodos. As classes de testes do JUnit seguem o padrão normal dos conceitos de classes.

Gabarito: Errado

7. (CESPE – ABIN– 2018) As ferramentas de execução do teste são classificadas como ferramentas de suporte para execução e registro e têm, como vantagem, o fato de não requererem um grande esforço para a obtenção de resultados expressivos.

Comentários: Pessoal, a questão erra ao afirmar: "As ferramentas de execução do teste são classificadas como ferramentas de suporte para execução e registro". Além disso, muitas vezes o tempo, o custo e o esforço para a implantação de uma ferramenta podem ser subestimados.

Gabarito: Errado

8. (Quadrix – CRM-PR – 2018) O JUnit é um método de testes de software do padrão xUnit voltado para sistemas desenvolvidos em linguagem Java.

Comentários: Pessoal, de fato, o JUnit é do padrão xUnit. xUnit.net é uma ferramenta de teste de unidade gratuita, de código aberto.

Gabarito: Correto

9. (CESPE – TRE BA– 2017) Uma equipe de desenvolvimento de projeto de automação comercial que padroniza os testes de software e, para isso, utiliza o JUnit para testar as classes Java desse projeto deve definir um método

- a) que não receba parâmetros
- b) público sem retorno de parâmetros
- c) público com retorno booleano.
- d) privado com retorno booleano.
- e) privado sem retorno booleano.

Comentários: Para utilizar o JUnit, é necessário criar uma classe que estenda junit.framework.TestCase. A partir daí, para cada método a ser testado é necessário definir métodos, os quais devem ser públicos e sem retorno de argumentos. Métodos testeXxx() na classe de teste devem ser public void e não podem receber nenhum parâmetro. Eles criam um objeto definindo o ambiente de teste, executam esses testes utilizando o ambiente que foi criado e verificam os

resultados que podem terminar, falhar ou provocar uma exceção. assim, como os Métodos testeXxx() na classe de teste devem ser public void e não podem receber nenhum parâmetro, nosso gabarito é a letra b) público sem retorno de parâmetros.

Gabarito: Letra B

10. (CESPE – FUNPRES-P-EXE– 2016) Na finalização de teste de software com o uso de JUnit, os dados de teste que tenham sido criados com JPA (Java Persistent API) são automaticamente removidos.

Comentários:

Pessoal, nada foi falado sobre remoção automática de dados criados com JPA feita com recursos próprios do JUnit. Essa questão está errada. O controle realizado pelo JUnit ocorre com as anotações @Before e @After que, respectivamente, permitem inserir ações que serão realizadas antes ou depois das chamadas aos métodos de teste.

Gabarito: Errado

11. (CESPE – FUNPRES-P-JUD– 2016) No JUnit, os testes são realizados em sequência, por isso eles mantêm uma relação de dependência entre si.

Comentários: Errado pessoal! Os testes são executados de forma isolada e independente.

Gabarito: Errado

12. (CESPE – AJ STJ– 2015) JUnit é um framework utilizado para facilitar a geração de testes a fim de se verificar se os resultados gerados pelos métodos escritos em Java são os esperados

Comentários: O JUnit facilita a criação e manutenção do código para a automação de testes com apresentação dos resultados. Com ele, pode ser verificado se cada método de uma classe funciona da forma esperada, exibindo possíveis erros ou falhas podendo ser utilizado tanto para a execução de baterias de testes como para extensão.

Gabarito: Correto

13. (CESPE – TJDF- 2015) Indica-se a automatização de testes para os testes de componentes e de sistema, visto que o uso de testes unitários é inviável, por dependerem de diversas possibilidades a serem avaliadas.

Comentários: Pessoal, na verdade, não é inviável o uso de testes unitários.

Gabarito: Errado

14. (CESPE – ATCG MEC – 2015) A JUnit é uma biblioteca típica de testes de apoio e fornece suporte para a execução de testes, geração de logs e verificação de resultados. Por meio do conjunto de classes C# do JUnit, é possível a criação de ambiente de testes automatizado.

Comentários: Vamos lembrar o conteúdo: O JUnit é um framework open-source, framework de teste de unidade para a linguagem de programação Java, criado por Erich Gamma e Kent Beck, com suporte à criação de testes automatizados na linguagem de programação Java. Ademais, deve-se destacar que um dispositivo de teste JUnit é um objeto Java. Portanto, não são classes C# e sim JÁVA!

Gabarito: Errado

15. (CESPE – ANATEL– 2014) Embora o JUnit apresente limitações, como a de não apoiar a aplicação de um critério de teste, esse framework pode ser utilizado ainda que estejam disponíveis apenas o bytecode e a especificação do programa.

Comentários: O JUnit tem diversas limitações, herdadas das ferramentas de teste de unidade em geral, como dificuldade de executar teste de GUI com eficiência, de escrever alguns testes para EJB, criar estatísticas de teste e escalabilidade de criação de casos de teste. O JUnit, porém, tem a capacidade de executar testes tanto em arquivos .java (não compilados) quanto em arquivos .class (classes compiladas Java, em bytecode).

Gabarito: Correto

16. (CESPE – ANATEL– 2014) Para a análise estática de código, a ferramenta Findbugs, que permite a verificação dos problemas relativos a defeitos e convenções de codificação em um software, é mais completa que a Checkstyle, que possibilita apenas a detecção dos problemas relativos a convenções de codificação.

Comentários: Pessoal, na verdade a questão erra ao dizer que o Findbugs permite a verificação dos problemas relativos a defeitos e convenções de codificação em um software. Pelo contrário, Findbugs não é focado em convenções de codificação em um software.

Gabarito: Errado

17. (CESPE – ANATEL– 2014) Por meio do JUnit, framework de teste que viabiliza a documentação e execução automática de testes de unidade em Java, é possível obter informações a respeito da cobertura obtida pelos casos de teste.

Comentários:

Questão errada. Para cobertura de casos de testes, é possível usar o EclEmma (EclEmma é uma ferramenta gratuita de cobertura de código Java para Eclipse, disponível sob a Licença Pública Eclipse. Ele traz a análise de cobertura de código diretamente para o ambiente de trabalho do Eclipse), ainda o Testes de mutantes (Em JÁVA é possível usar ferramentas como JaCoCo, Cobertura e JCov.)

Gabarito: Errado

18. (CESPE – FUB – 2013) O desenvolvimento de software direcionado a testes pode contar com o uso de ferramentas automatizadas para criação de testes, como, por exemplo, o JUnit, um framework para testes em Java.

Comentários:

Pessoal, perfeito! É exatamente isso! como já foi falado na aula, o JUnit, um framework para testes em Java.

Gabarito: Correto

19. (CESPE – BRB– 2011) JUnit é um framework open source que realiza testes unitários funcionais e de integração em aplicações desenvolvidas em qualquer linguagem.

Comentários:

Pessoal, o JUnit é um framework open-source, framework de teste de unidade para a linguagem de programação Java. Não em qualquer linguagem.

Gabarito: Errado

20. (FGV – TJ AM – 2013 – Adaptada) JUnit constitui um conjunto de classes em Java, que o usuário estende para criar um ambiente de testes automatizado.

Comentários:

pessoal, perfeito! JUnit é um framework, inicialmente criado por Kent Beck e Erich Gamma, que se tornou um padrão de fato para testar unidades de aplicações java. Uma das vantagens do uso do framework JUnit é a possibilidade de realizar testes automatizados, executando um conjunto de testes de uma só vez, o que permite a realização de testes de regressão.

Gabarito: Correto

LISTA DE QUESTÕES - JUNIT

1. **(CESPE – DP DF– 2022)** Julgue o item a seguir, acerca de CSS3, JMS, JSON e JUnit.

Na definição de métodos de teste em JUnit, a anotação `@BeforeClass` pode ser usada em métodos que implementem atividades que consomem muito tempo.

2. **(CESPE – PGDF– 2021)** Julgue o item seguinte, a respeito de JMS (Java Message Service), JUnit e Spring Framework.

O JUnit 5 é formado por JUnit Platform, JUnit Jupiter e JUnit Vintage; o JUnit Jupiter pode ser utilizado em programas escritos em Kotlin.

3. **(CESPE – TJ PA– 2020)** Assinale a opção que apresenta a anotação que pode ser utilizada no JUnit para condicionar a execução de um teste a partir da avaliação de um script escrito em JavaScript.

- a) `@EnabledOnOs`.
- b) `@EnabledOnJre`
- c) `@DisabledIfSystemProperty`.
- d) `@DisabledIfEnvironmentVariable`
- e) `@EnabledIf`

4. **(CESPE – COGE CE – 2019)** Nos testes unitários, é possível testar uma classe ou até mesmo um objeto Java. Nesse contexto, uma classe, para ser testada pelo Junit, deve se estender a partir de

- a) `Collection`.
- b) `TestExecution`.
- c) `TestSetUp`.
- d) `TestCase`
- e) `TestReporter`.

5. **(CESPE – SLU-DF – 2019)** Na prática de integração contínua para desenvolvimento de software, vários colaboradores criam e mantêm o código de forma organizada e controlada, utilizando ferramentas como Git (controle de versão), Junit (testes), Hudson e Jenkins (deploys em ambientes de desenvolvimento e produção), o que reduz a geração de erros de integração.

6. **(CESPE – TJ STJ– 2018)** Uma característica e limitação do JUnit é a impossibilidade de definição de parâmetros para construtores e métodos.

7. **(CESPE – ABIN– 2018)** As ferramentas de execução do teste são classificadas como ferramentas de suporte para execução e registro e têm, como vantagem, o fato de não requererem um grande esforço para a obtenção de resultados expressivos.
8. **(Quadrix – CRM-PR – 2018)** O JUnit é um método de testes de software do padrão xUnit voltado para sistemas desenvolvidos em linguagem Java.
9. **(CESPE – TRE BA– 2017)** Uma equipe de desenvolvimento de projeto de automação comercial que padroniza os testes de software e, para isso, utiliza o JUnit para testar as classes Java desse projeto deve definir um método
- a) que não receba parâmetros
 - b) público sem retorno de parâmetros
 - c) público com retorno booleano.
 - d) privado com retorno booleano.
 - e) privado sem retorno booleano.
10. **(CESPE – FUNPRES-P-EXE– 2016)** Na finalização de teste de software com o uso de JUnit, os dados de teste que tenham sido criados com JPA (Java Persistent API) são automaticamente removidos.
11. **(CESPE – FUNPRES-P-JUD– 2016)** No JUnit, os testes são realizados em sequência, por isso eles mantêm uma relação de dependência entre si.
12. **(CESPE – AJ STJ– 2015)** JUnit é um framework utilizado para facilitar a geração de testes a fim de se verificar se os resultados gerados pelos métodos escritos em Java são os esperados
13. **(CESPE – TJDF- 2015)** Indica-se a automatização de testes para os testes de componentes e de sistema, visto que o uso de testes unitários é inviável, por dependerem de diversas possibilidades a serem avaliadas.
14. **(CESPE – ATCG MEC – 2015)** A JUnit é uma biblioteca típica de testes de apoio e fornece suporte para a execução de testes, geração de logs e verificação de resultados. Por meio do conjunto de classes C# do JUnit, é possível a criação de ambiente de testes automatizado.
15. **(CESPE – ANATEL– 2014)** Embora o JUnit apresente limitações, como a de não apoiar a aplicação de um critério de teste, esse framework pode ser utilizado ainda que estejam disponíveis apenas o bytecode e a especificação do programa.
16. **(CESPE –ANATEL– 2014)** Para a análise estática de código, a ferramenta Findbugs, que permite a verificação dos problemas relativos a defeitos e convenções de codificação em um software, é mais completa que a Checkstyle, que possibilita apenas a detecção dos problemas relativos a convenções de codificação.

- 17. (CESPE – ANATEL– 2014)** Por meio do JUnit, framework de teste que viabiliza a documentação e execução automática de testes de unidade em Java, é possível obter informações a respeito da cobertura obtida pelos casos de teste.
- 18. (CESPE – FUB – 2013)** O desenvolvimento de software direcionado a testes pode contar com o uso de ferramentas automatizadas para criação de testes, como, por exemplo, o JUnit, um framework para testes em Java.
- 19. (CESPE –BRB– 2011)** JUnit é um framework open source que realiza testes unitários funcionais e de integração em aplicações desenvolvidas em qualquer linguagem.
- 20. (FGV – TJ AM – 2013 – Adaptada)** JUnit constitui um conjunto de classes em Java, que o usuário estende para criar um ambiente de testes automatizado.

GABARITO

1. CORRETO
2. CORRETO
3. LETRA E
4. LETRA D
5. CORRETO
6. ERRADO
7. ERRADO
8. CORRETO
9. LETRA B
10. ERRADO
11. ERRADO
12. CORRETO
13. ERRADO
14. ERRADO
15. CORRETO
16. ERRADO
17. ERRADO
18. CORRETO
19. ERRADO
20. CORRETO

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.